lifecycle生命周期

Vue2.x	Vue3.x
beforeCreate	Not needed*
created	Not needed*
beforeMount	onBeforeMount
mounted	onMounted
beforeUpdate	onBeforeUpdate
updated	onUpdated
beforeUnmount	onBeforeUnmount
unmounted	onUnmounted

```
<template><button @click="count++">{{count}}</button></template>
setup(){// 组合式API的入口函数，在beforeCreate之前执行
        const count=ref(0);
        console.log('---setup---')
          onBeforeMount(()=>{//组件挂载到节点之前执行
            console.log('---onBeforeMount---开始执行了-----')
          })
          onMounted(()=>{//组件挂载完成
            console.log('---onMounted---开始执行了-----')
          })
          onBeforeUpdate(()=>{//组件更新前执行
            console.log('---onBeforeUpdate---开始执行了-----')
          })
          onUpdated(()=>{//组件更新完成后执行
            console.log('---onUpdated---开始执行了-----')
          })
          onBeforeUnmount(()=>{//组件卸载之前执行
            console.log('---onBeforeUnmount---开始执行了-----')
          })
          onUnmounted(()=>{//组件卸载完成后执行
            console.log('---onUnmounted---开始执行了-----')
          })
          return{count}
        }
```

provide

provide ：向子组件以及子孙组件传递数据。接收两个参数，第一个参数是 key，即数据的名称；第二个参数为 value，即数据的值。

query

inject

inject ：接收父组件或祖先组件传递过来的数据。接收一个参数 key，即父组件或祖先组件传递的数据名称

```
<template>//father组件
    <div style="border: red 2px solid; margin: 5px;">
        <input type="text" v-model="str">
        <One></One>
    </div>
</template>
    import {provide,inject,ref} from 'vue'
    import One  from './One.vue'
    export default{
        components:{
            One
        },
        setup(){
            // father作为父级组件，通过provide函数 提供数据共享 (不限层次),
            // 注意: provide 只能向下传递数据，向子孙级别组件共享数据
            //值一数据别名，值二数据变量名、字符串等等
            provide('globalVal','这是father组件传递过来的值')
            const  str= ref('str')
            provide('globalStr',str)
            return{
                str
            }
        }
    }

<template>//one组件
    <div style="border: orange 2px solid; margin: 5px;">
        <h2>One组件</h2>
        <Two></Two>
    </div>
</template>
    import {inject,provide} from 'vue'
```

```
import Two  from './Two.vue'
export default{
    components:{
        Two
    },
    setup(){
      const val=inject('globalVal');//接收父组件传来的值
      const str=inject('globalStr');
      return{
          val,
          str
      }
    }
}


<template>//two组件
    <div style="border: green 2px solid; margin: 5px;">
    </div>
</template>
    import {inject} from 'vue'
    export default{
        setup(){// inject 遵循就近原则
          // inject ,通过自定的函数名称获取到父子级组件的共享数据
          const val=inject('globalVal');//接收祖父组件传来的值
          const str=inject('globalStr')
          return{
              val,
              str
          }
        }
    }
```

自定义hooks

抽离出来单独功能的函数

创建一个js文件，然后暴露出去

在要调用的组件中引入

```
//js文件
import {reactive,toRefs,computed} from 'vue'
function userAdd() {}
export default userAdd
//组件
import userAdd from 'XXXXXX'
    export default {
        setup(){//解构方法
```

```
        const {total,val,todos,addUser} = userAdd();
        //同样需要返回才能在template中使用
        return {total,val,todos,addUser,}
    }
}
```